

slugify npm v1.6.6 1	coverage 100% 2	<pre>var slugify = require('slugify') slugify('some string') // some-string // if you prefer something other than '-' as separator slugify('some string', '_') // some_string</pre> - Vanilla ES2015 JavaScript - If you need to use Slugify with older browsers, consider using version 1.4.7 3	- No dependencies - Coerces foreign symbols to their English equivalent (check out the charMap for more details) - Works in the browser (window.slugify) and AMD/CommonJS-flavored module loaders 4
8 The main charmap.json file contains all known characters and their transliteration. All new characters should be added there first. In case you stumble upon a character already set in charmap.json, but not transliterated correctly according to your language, then you have to add those characters in Locales If the value of remove is a string, it should be a single character. Otherwise, the remove option might not work as expected.	7 For example, to remove *+~.()''"':@ from the result slug, you can use slugify('...', {remove: /[*+~.()''"':@]/g}). If the value of remove is a regular expression, it should be a character class and only a character class. It should also use the global flag. (For example: /[*+~.()''"':@]/g.) Otherwise, the remove option might not work as expected. Remove	9 <pre>strict: false, // strip special characters except replacement, locale: 'vi', // language code of the locale to use trim: true // trim leading and trailing replacement chars, defaults to `true` })</pre>	5 <pre>slugify('some string', { replacement: '-', // replace spaces with ' ', // replace character, defaults to `'-` remove: undefined, // remove characters that match regex, defaults to `undefined` lower: false, // convert to lower case, defaults to `false` })</pre> Options
9 locales.json to override the already existing transliteration in charmap.json, but for your locale only. You can get the correct language code of your language from here. Extend Out of the box slugify comes with support for a handful of Unicode symbols. For example the ☢ (radioactive) symbol is not defined in the charMap and therefore it will be stripped by default:	10 <pre>slugify('unicode ♥ is ☢') // unicode- love-is</pre> However you can extend the supported symbols, or override the existing ones with your own: <pre>slugify.extend({'☢': 'radioactive'}) slugify('unicode ♥ is ☢') // unicode- love-is-radioactive</pre> Keep in mind that the extend method extends/overrides the default charMap for the	11 entire process. In case you need a fresh instance of the slugify's charMap object you have to clean up the module cache first: <pre>delete require.cache[require.resolve('slugify')] var slugify = require('slugify')</pre> Contribute - Add chars to charmap.json - Run tests npm test - The tests will build the charmap in index.js and will sort the charmap.json	12 4. Commit all modified files Originally this was a vanilla javascript port of node-slug. Note that the original slug module has been ported to vanilla javascript too.
11	13	14	13